Services, WorkManager, Build & release process

Android Lecture 5

Simona Kurňavová

Today’s Agenda

- Services
- Work Manager
- Build process
- Release process
- Q&A

Services

Services

- Documentation
- Long running operation in background
- Not bound with UI
- Can expose API for other applications
- By default runs on UI thread
- **Types**: Foreground, Background, Started, Bound

Services

- Needs to be declared in manifest:

AndroidManifest.xml

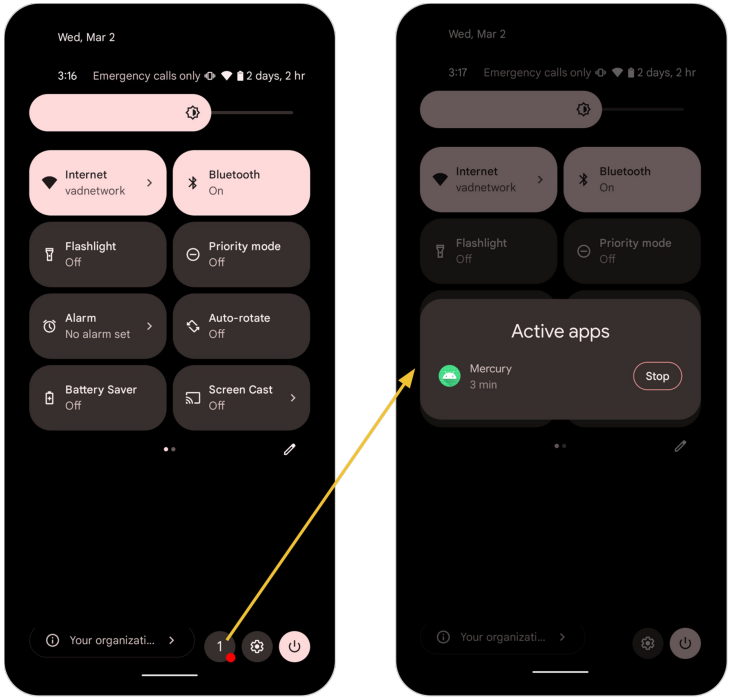
```
<manifest ... >
  ...
  <application ... >
      <service android:name=".ExampleService" />
      ...
  </application>
</manifest>
```

Background service

- On background by default
- Strongly limited since Android Oreo (API 26)
  - Not possible to start background service when app is not on the foreground
  - Details in documentation
- *Example usage*: Sync against backend
- Alternatively use WorkManager

Foreground service

- Service process has higher priority
- User is actively aware of it – requires **permanent notification**
  - Notification shown under Ongoing header
  - Lower than Android 13: notification cannot be dismissed
  - Android 13 and higher: notification dismissible by default
- System not likely to kill foreground services
- Apps targeting Android 9 (API 28) or higher must define
    - FOREGROUND_SERVICE permission (normal permission)
- *Example usage*: Downloading files
- Alternatively use foreground Workers



Started Service

Service that is started, runs independent of UI and does not return result to caller.

Example usage: started from Activity in order to save data to database.

Starting:

- Started by another component calling `startService()`
- When started, it's lifecycle is independent from the caller

Stopping:

- **System will not stop started service**
- Service can stop itself: `Service#stopSelf()`
- Other component can stop the service: `Context#stopService()`

Started Service

Starting of Service

```
context.startService(Intent(context, HelloService::class.java))
```

Must implement onStartCommand

```
class ExampleService : Service() {
    ...

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        // The service is starting, due to a call to startService()
        return startMode
    }

    ...
}
```

Bound service

Service is bound when another component calls bindService() and offers client-server interface for communication.

Example usage: Music player service with controls in Activity.

Binding:

- Component calls bindService()
- Service overrides Service#onBind(intent: Intent): IBinder?
- IBinder: Object providing API to communicate with service

Unbinding:

- Client calls unbindService()

Bound service

Bound Service implementation

```
class LocalService : Service() {

    private val binder = LocalBinder() // Binder given to clients.

    /** Method for clients.  */
    val randomNumber: Int
        get() = Random().nextInt(100) // Generate random number

    /**
     * Class used for the client Binder.
     */
    inner class LocalBinder : Binder() {
        // Return this instance of LocalService so clients can call public methods.
        fun getService(): LocalService = this@LocalService
    }

    override fun onBind(intent: Intent): IBinder {
        return binder
    }
}
```

Bound service: ServiceConnection

Client needs to implement ServiceConnection callbacks

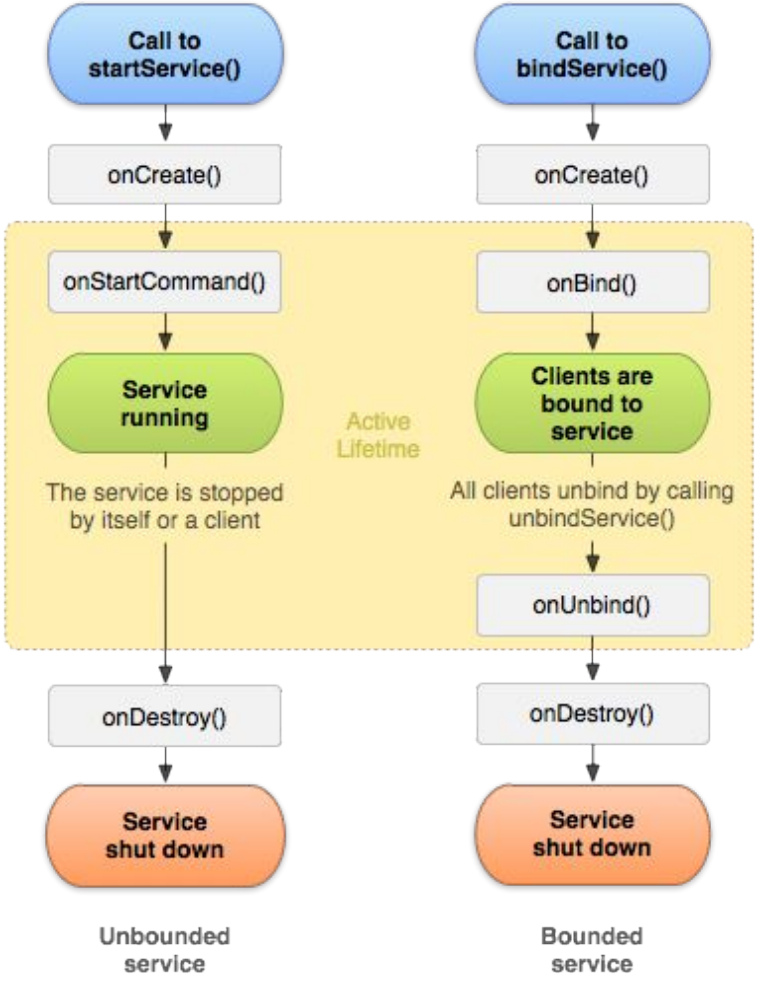
ServiceConnection implementation

```
private val connection = object : ServiceConnection {

    override fun onServiceConnected(className: ComponentName, service: IBinder) {
        // Connection with the service has been established.
        val binder = service as LocalService.LocalBinder
        val service = binder.getService()
        val randomNum = service.randomNumber
        ...
    }

    override fun onServiceDisconnected(arg0: ComponentName) {
        // Service was disconnected.
        // Process hosting service crashed or been killed
        // Service connection remain active (onServiceConnected can be called again)
    }

    override fun onBindingDied(name: ComponentName?) { // Optional impl
        // Binding is dead
        // Can happen during app update
        // Unbind and rebind
    }

    override fun onNullBinding(name: ComponentName?) { // Optional impl
        // Service#onBind returns null
        // Unbinding is still required
    }
}
```

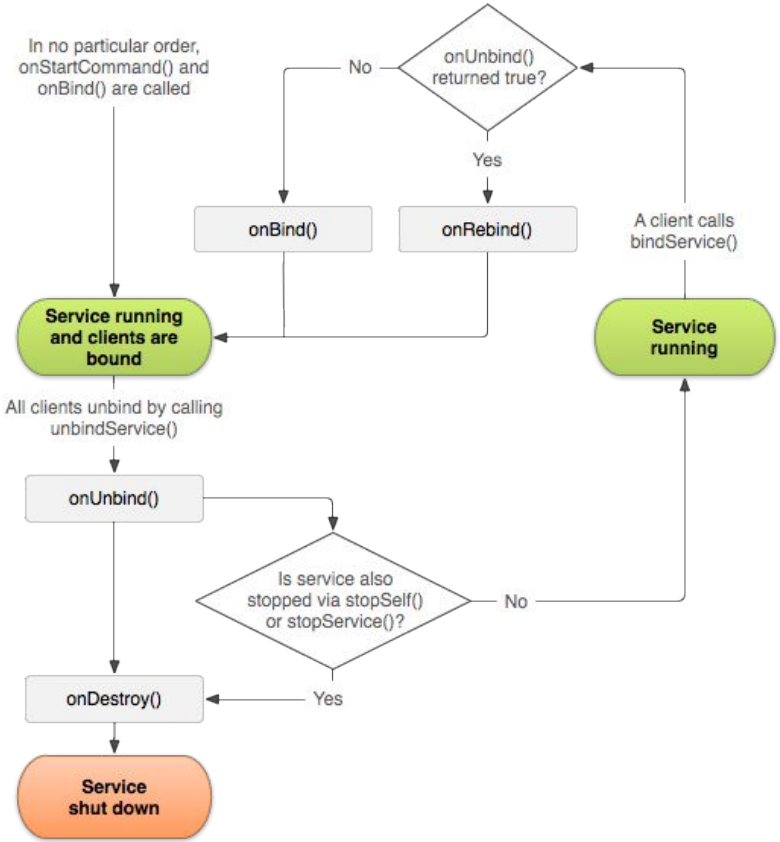
Lifecycle of Service

onStartCommand:

- Intent as parameter
- Return value specifies behaviour what to do when it is killed by system (not recreated, recreated with same intent, recreated with null intent)



Lifecycle of a hybrid Service (started + bound)



WorkManager

WorkManager

- Documentation
- Backward compatible up to API 14
- Ensure execution even if app or device restarts
- Adheres to doze mode
- **Uses**:
  - JobScheduler on devices with API 23+
  - Combination of BroadcastReceiver + AlarmManager API 14-22

WorkManager

- One-off or periodic
- Monitor and manage scheduled tasks
- Chain tasks
- Observable status
- **Work constraints**
  - Network
  - Charging status
  - ...

Workers

```
class UploadWorker(appContext: Context, workerParams: WorkerParameters) : Worker(appContext, workerParams) {

    override fun doWork(): Result {
            // Get the input
            val imageUriInput = getInputData().getString(Constants.KEY_IMAGE_URI)

            // Do the work
            val response = uploadFile(imageUriInput)

            // Create the output of the work
            val outputData = workDataOf(Constants.KEY_IMAGE_URL to response.imageUrl)

            // Return the output
            return Result.success(outputData)
    }
}
```

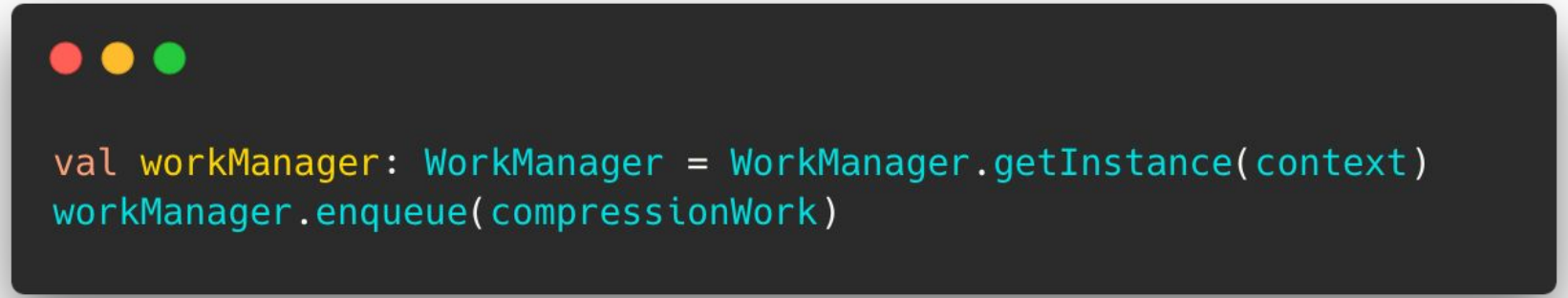
Work requests

```
// Create a Constraints object that defines when the task should run

val constraints = Constraints.Builder()
        .setRequiresDeviceIdle(true)
        .setRequiresCharging(true)
        .build()

// ...then create a OneTimeWorkRequest that uses those constraints

val compressionWork = OneTimeWorkRequestBuilder<CompressWorker>()
        .setConstraints(constraints)
        .build()
```

https://developer.android.com/topic/libraries/architecture/workmanager/how-to/define-work

Enqueue Worker

- **enqueue**()
- **enqueueUniqueWork**()
- **enqueueUniquePeriodicWork()**



```
val workManager: WorkManager = WorkManager.getInstance(context)
workManager.enqueue(compressionWork)
```

Foreground Workers

- Long-running workers
- Can run longer than 10 minutes
- Need to show notification
- Uses foreground service

```
class UploadWorker(appContext: Context, workerParams: WorkerParameters) : Worker(appContext, workerParams) {
    override fun doWork(): Result {
            // Mark worker as long-running/important
            setForeground(ForegroundInfo(notificationId, getNotification()))
            ...
            return Result.success()
    }
}
```

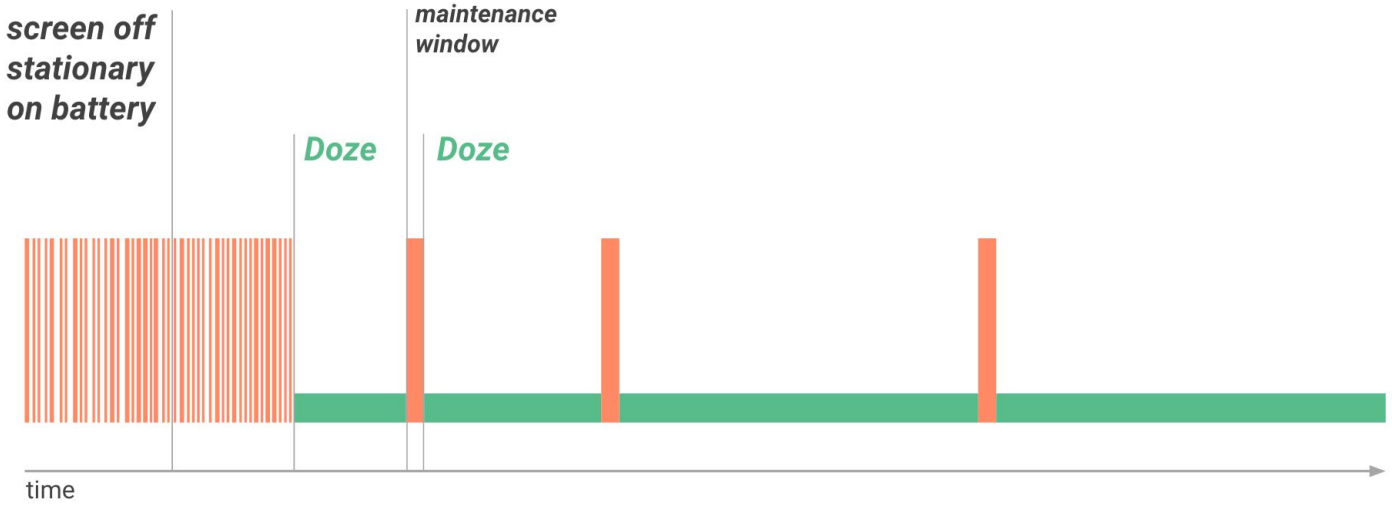
Doze Mode

Doze mode

- Documentation
- Power saving feature
- Introduced in Android 6
- Restrict app access to network and cpu intensive services
- Defers jobs, syncs and alarms

Doze mode

- *Maintanance window*: periodical exits of "Doze mode" and allows apps to complete their deffered tasks
- Over time, system schedules maintanance windows less frequently



Alarm Manager

Alarm manager

- Performs time-based operations outside the application lifecycle
- Fire intents at specified time
- Operate outside of your application, trigger events or actions even when app is not running or device is asleep
- Action is specified by PendingIntent
- In conjunction with broadcast receivers start services
- Many API changes over Android versions:
  - Added some new method
  - Some method changed behaviour from exact -> inexact
  - read the documentation carefully before usage!

Alarm manager: Alarm types

- Clock types
  - Elapsed: time since system boot (use when there is no dependency on timezone)
  - Real time clock: time since epoch (use when you need to consider timezone/locale)
- Wake up
  - Wakeup: ensure alarm will fire at the scheduled time
  - Non-wakeup: alarm are fired when device awakes

Constants:

- ELAPSED_REALTIME
- ELAPSED_REALTIME_WAKEUP
- RTC
- RTC_WAKEUP

AlarmManager: Usage

```
val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager

val intent = Intent("AlarmAction")

val pendingIntent = PendingIntent.getBroadcast(
  applicationContext,
  ALARM_REQUEST_CODE,
  intent,
  PendingIntent.FLAG_UPDATE_CURRENT
)

alarmManager.set(AlarmManager.RTC,
                 System.currentTimeMillis() + TimeUnit.HOURS.toMillis(1L),
                 pendingIntent)
```

- AlarmType
- Time: Depending on the alarm type it is timestamp or time since device boots
- PendingIntent: PendingIntent which specify action which should happen

Alarm manager: Sleeping device

Alarm manager can wake sleeping device and triggered pending intent is able to start activity/service or send broadcast.

- Problem: it is not guaranteed by system to start service/activity before device falls asleep again. Only *BroadcastReceiver.onReceive* is guaranteed to keep device awake. If you start activity/service in receiver, there is no guarantee that activity/service will start before the wake lock is released.
- Solution: **Wake locks**

Wake locks

- Prevent app from sleeping
- Requires permission android.permission.WAKE_LOCK
- Multiple levels
  - PARTIAL_WAKE_LOCK: CPU is running, screen and keyboard backlight allowed to go off
  - FULL_WAKE_LOCK: Screen and keyboard on full brightness, released when user press power button
  - SCREEN_DIM_WAKE_LOCK: Screen is on, but can be dimmed, keyboard backlight allowed to go off, released when user press power button
  - SCREEN_BRIGHT_WAKE_LOCK: Screen on full brightness, keyboard backlight allowed to go off, released when user press power button

Wake locks

- Example of obtaining wake lock:

```
val wakeLock: PowerManager.WakeLock =
        (getSystemService(Context.POWER_SERVICE) as PowerManager).run {
            newWakeLock(PowerManager.PARTIAL_WAKE_LOCK, "MyApp::MyWakelockTag").apply {
                acquire()
            }
        }
…

wakeLock.release()
```

Alarm manager: Sleeping device

Alarm manager can wake sleeping device and triggered pending intent is able to start activity/service or send broadcast.

- Problem: it is not guaranteed by system to start service/activity before device falls asleep again. Only *BroadcastReceiver.onReceive* is guaranteed to keep device awake. If you start activity/service in receiver, there is no guarantee that activity/service will start before the wake lock is released.
- Solution: Acquire your wake lock during BroadcastReciver.onReceive and before starting service -> start service -> when service finish its job release the wake lock (it is really important to release wake lock, it disables turning off the CPU).

Alarm manager & WorkManager tips

- For synchronization consider to use WorkManager
- Alarms are cancelled on reboot, reschedule alarms when device boots
- When syncing against backend - distribute this over time, not schedule each sync to be on specified time
  - Imagine 1M+ of devices trying to download something from your server at the same time

Permissions

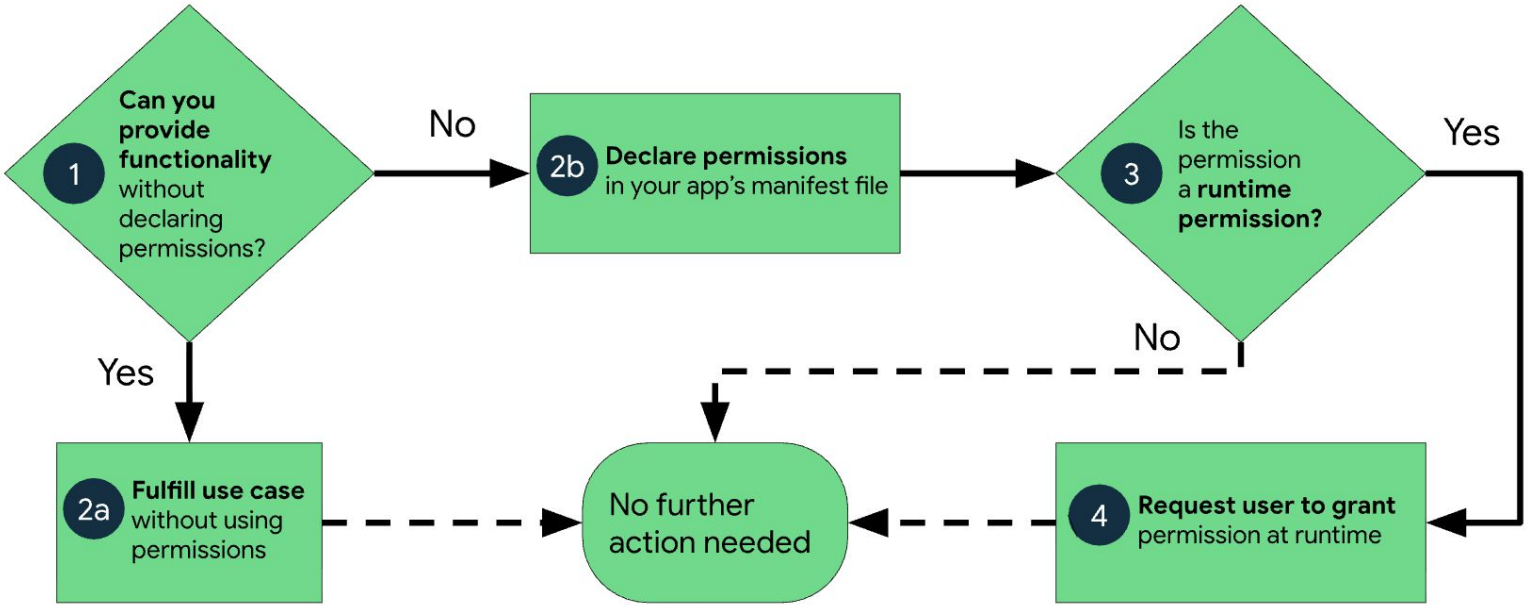
Permissions

- Documentation
- Security mechanism to restrict access to user information & data
- Why:
  - Prevent access to sensitive data
  - Restricts actions the app can do
  - User is aware about what apps can and cannot do - is provided rational for sensitive permissions

Permissions: Types

- Normal permissions
  - Needs to be declared in the Manifest
  - Granted on install
  - User can view these permissions in Google Play listing
  - Do not require explicit user consent (dialog/screen with explanation) and user cannot deny them
  - Example: INTERNET, ACCESS_NETWORK_STATE
- Runtime permissions
  - Dangerous permissions granting more access to data and actions
  - Also needs to be declared in Manifest
  - Requires explicit consent from user - Needs to trigger system dialog that user can confirm
  - They can be denied
  - Example: CAMERA

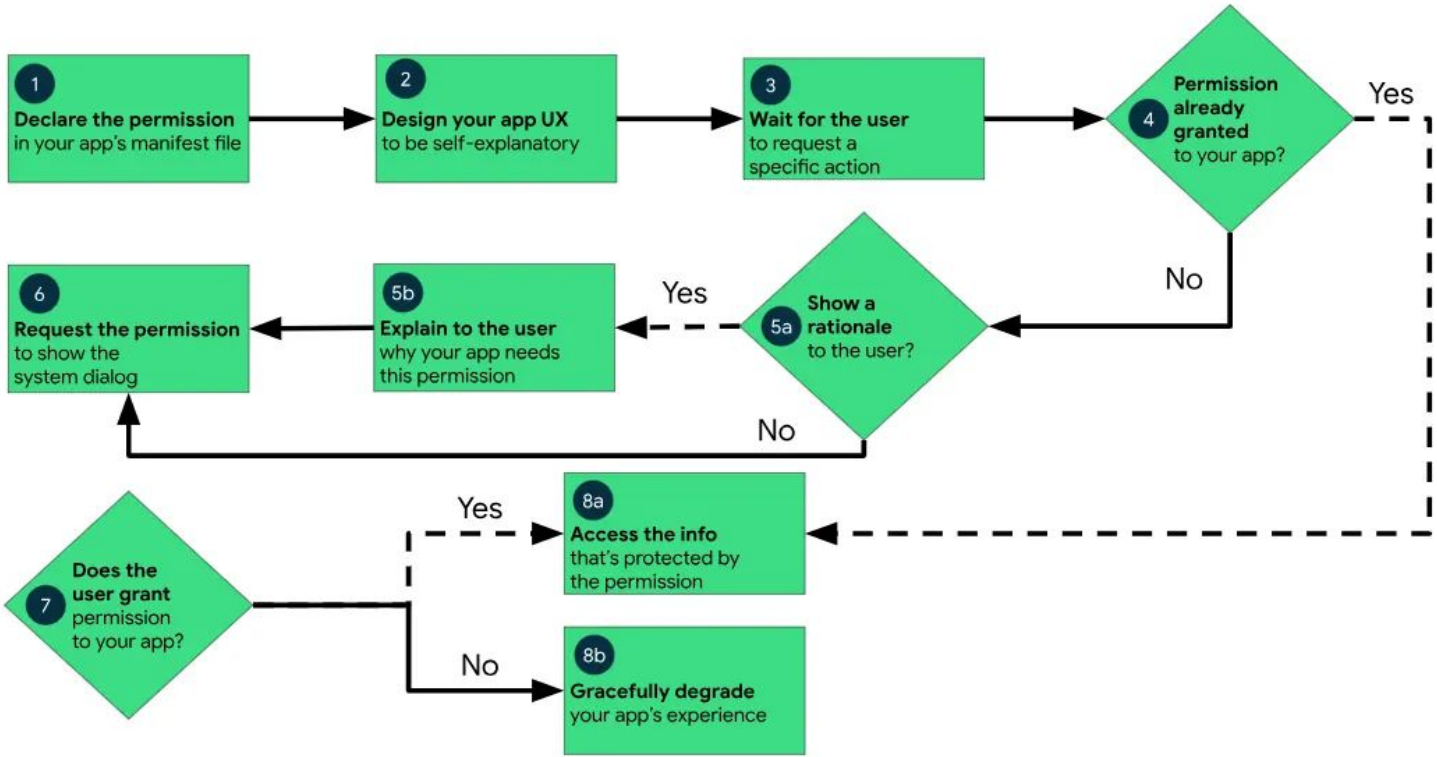
Permissions: Workflow of using permissions



Permissions: Types

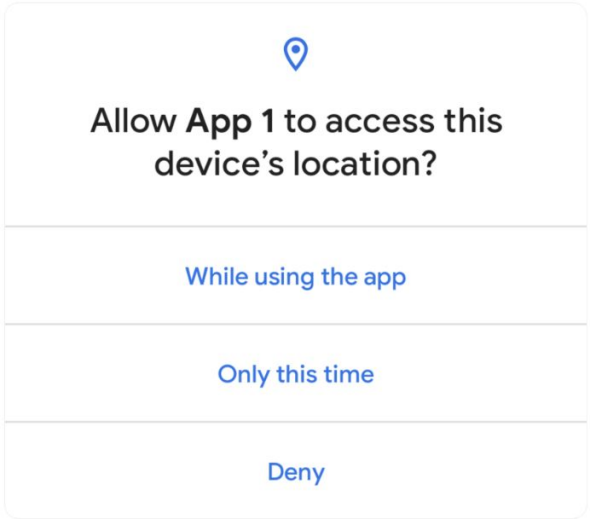
- Special permissions
  - Usually user needs to allow special permissions inside settings - the app should lead them there
  - More sensitive permissions
  - Requires justification (clear explanation why it is needed)
  - Example: Access location in background (ACCESS_BACKGROUND_LOCATION)

Permissions: How to ask for special permission



Permissions: One-time permissions

- Documentation
- Introduced in Android 11
- Temporary permission meant for single use
- Often used for camera or location



Permissions: Declaring

Declaring permission in Manifest

```
<uses-permission android:name="android.permission.CAMERA"/>
```

Permissions: Common permissions

- Internet permission `INTERNET`
- Notification permission (runtime/special) `POST_NOTIFICATIONS`
- Location Permission (runtime) `ACCESS_COARSE_LOCATION`, `ACCESS_FINE_LOCATION`, `ACCESS_BACKGROUND_LOCATION`
- Camera Permission (runtime)
- Schedule exact alarms (special) `SCHEDULE_EXACT_ALARMS`

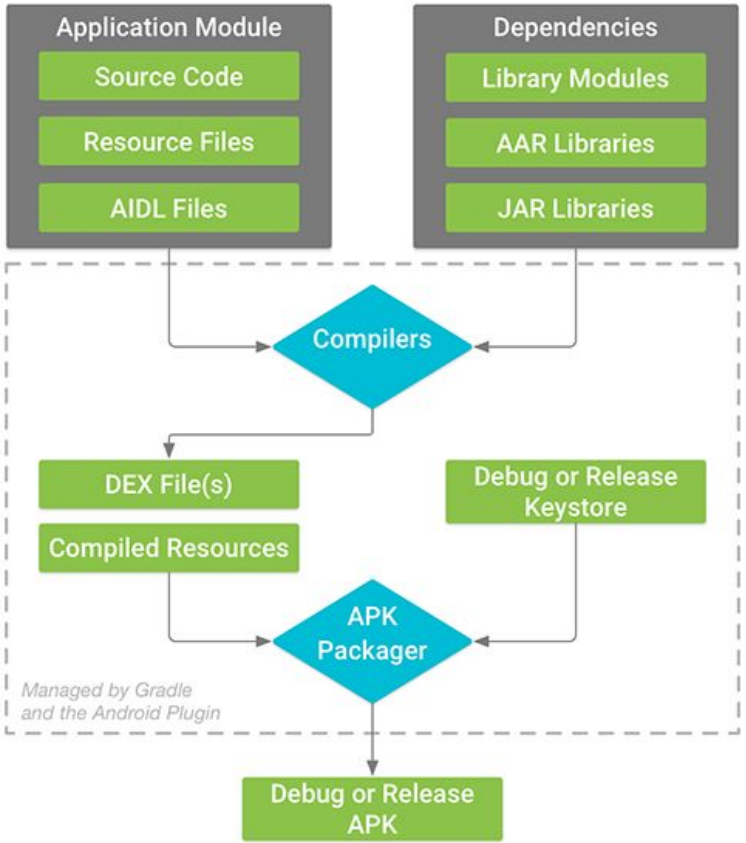
Permissions: Good Practice

- Documentation
- Ask for runtime permission only before you need it
- Explain to user why you need the permission before showing system dialog
- Do not assume permission was already granted - always check it before attempting the action
- Make sure to support all Android version (from minSDK to targetSDK) - permissions often change with new API
  - User can always disable permission in settings
  - For example: Notification permission was not required before API 33 (Android 13)
- Remove from app any permission that is not needed
- If permission is denied - explain to user implications of this denial

Build Process

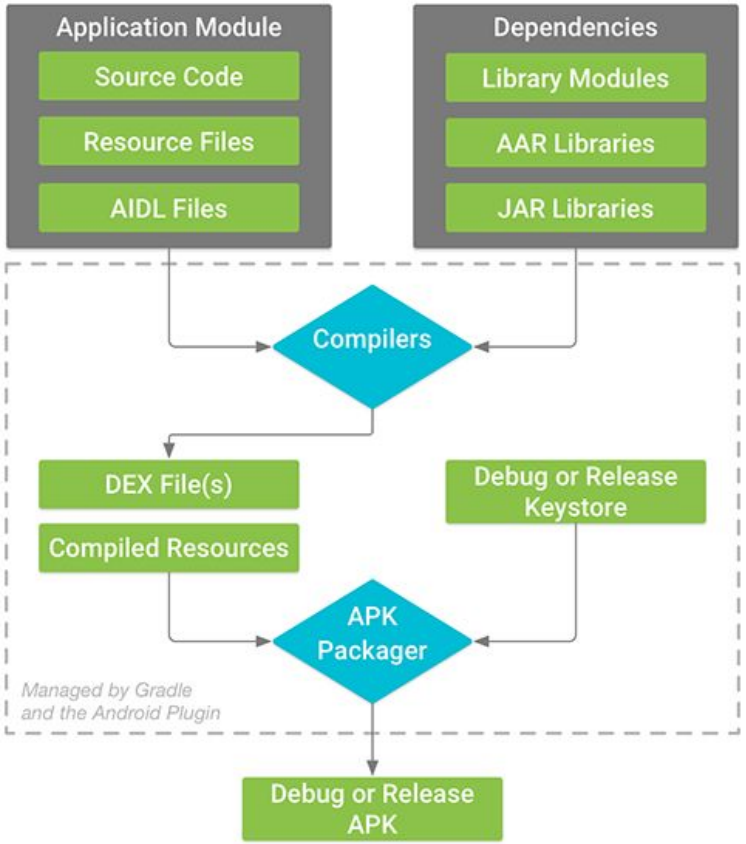
Build process: Simplified

1. Kotlin and Java is compiled to .class files
2. KAPT/KSP processing: Hilt, Room, etc. annotations generate more .class files
3. All .class files are converted to DEX (Dalvik Executable) files (so that they be able to run on Android)
    a. Multidex files - contain method references



Build process: Simplified

4. Resource processing and packaging
   - R.java class is generated by AAPT2 tool with ids of resources
5. Manifest merging from all modules and parts of app
6. Shrinking and obfuscation



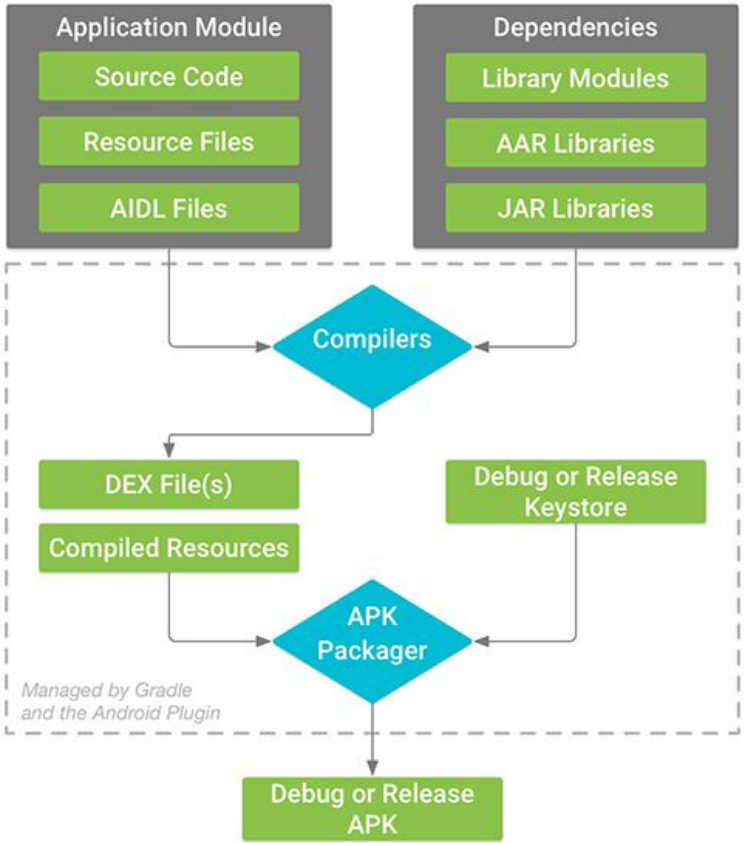
Build process: Simplified

7. Packaging: compiled code, resources, manifest, libraries, etc.

Result: apk or aab (app bundle)

8. Signing

- Debug builds = auto-signed.
- Release builds = your keystore that Gradle uses to sign apk



Bundle

- It is not directly installable
- contains everything needed to generate apk optimised for specific device
  - all app architectures
  - all screen densities resources
  - …
- required by Google Play since 2021
- Benefits: smaller downloads for users, dynamic features (download only when needed)

Release Process

Before release

- Make sure debug logs are not present in the final build
- Define version of the application (only higher versions can be uploaded)
- Review permissions, unused resources/assets
- Make sure the app is aligned with Google policy
- Ensure backend is ready & running (if needed)

Testing: Why test

- Prevent bugs from reaching users
- Maintain quality of application
- Gain valuable feedback from testers (or yourself from user pov)
- Ensure app stability
- Asses risks
- Safe money and time

Testing: Types of tests

- End-to-End / Smoke
  - testing of whole big flows, simulating standard user actions
- Regression
  - avoiding reintroducing previous bugs; sanity check of old logic
- Integration
  - integration of new components/features and how it works in a system
- Performance/load/stess
  - big data, high network trafic, many users, etc.

Testing: Automation

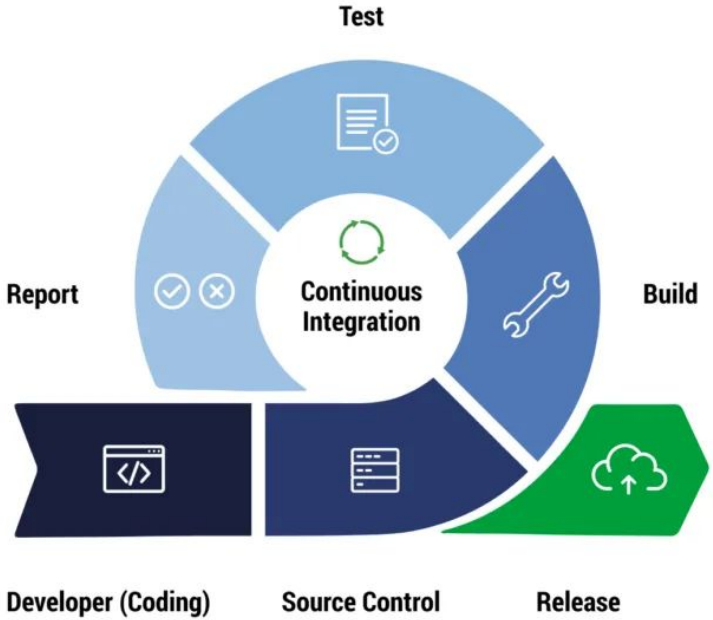
- Unit tests
  - Each test specialised for one unit - class, method, component, etc.
  - Unit should be independent from other components if possible
  - *Technology*: JUnit (test runner), Mockito, Mockk, Asserts (Truth, AssertJ, etc.)
- Instrumentation tests
  - Used when Android runtime environment is needed
  - Usually UI tests or tests of Android specific components - database, Work Manager, etc.
  - Runs on emulator
- Screenshot/Snapshot testing
  - Can be used for regression, smoke testing, but also as a documentation of designs
  - *Technology*: Paparazzi, Roborazzi, Compose UI test, …
- UI tests
  - Tests for UI components or flow
  - Run as Instrumentation tests
  - Verify if correct text/buttons/components are present under specific conditions
  - Can also verify that given action triggers the right UI reaction and navigation works as expected
  - *Technology*: UiAutomator, Espresso

Testing: Static code analysis

- Tools that detect problems in the code:
  - formatting and code style
  - using specified language constructs
  - Number of methods per interface or class, number of lines per method, etc.
  - Unused resources
- *Why use them:*
  - maintenance of quality of code
  - avoidance of common error
  - unified code style
- *Android tools*: detekt, lint, etc.

Continuous integration (CI)

- Building of apk/bundle
- Test Automation
  - running daily/nightly, after every new commit, manual trigger, etc.
- Automation of release/deploy process
- Examples: Teamcity, Jenkins, Bitrise, Gitlab, …



Google Play Release

- Developer account is needed (25 USD)
- Release types:
  - Internal testing
  - Alpha (open testing)
  - Beta
  - Production
- *Staged rollout* - start at small percentage, increase progressively
- Release notes - public vs. internal
- Google Play listing: screenshots, texts, pricing

Release monitoring

- Possibility of crashes, ANR, user complains
- Tools: Google Play Monitoring, Firebase Crashlytics, other analytics and logging tools
- User review
- Backend traffic (load, changes in traffic)
- Number of users, version distribution, marketing statistics, etc.
- In case of problem -> **halt and hotfix**

Thank you for attention

Feel free to leave feedback about this lesson: